PostgreSQL Relationships — Interview Practice

10 combined SQL problems • Easy → Medium → Hard • Logic-focused

Topics Covered

Concept	Practice Focus
Primary Key / Foreign Key	Referential relationships between tables
One-to-One	Matching records across uniquely related tables
One-to-Many	Parent-child data relationships
INNER JOIN	Only matching records
LEFT JOIN	All records from the left table + matches
RIGHT JOIN	All records from the right table + matches
CROSS JOIN	All possible combinations
FULL JOIN	Matched + unmatched records from both sides

Practice Database Schema

departments
department_id INT PRIMARY KEY
department_name VARCHAR(50)

employees
employee_id INT PRIMARY KEY
employee_name VARCHAR(100)
department_id INT REFERENCES departments(department_id)
manager_id INT REFERENCES employees(employee_id)
salary NUMERIC(10,2)

employee_details — one-to-one with employees
employee_id INT PRIMARY KEY REFERENCES employees(employee_id)
phone VARCHAR(15)
email VARCHAR(100)
city VARCHAR(50)

projects
project_id INT PRIMARY KEY
project_name VARCHAR(100)
department_id INT REFERENCES departments(department_id)

employee_projects — connects employees and projects
employee_id INT REFERENCES employees(employee_id)
project_id INT REFERENCES projects(project_id)
hours_worked INT

Assume realistic sample data exists in all tables. Write SQL queries only; do not create or insert data unless a question explicitly asks you to.

Easy Level

Easy 1 — Basic INNER JOIN

Display each employee's **employee_id**, **employee_name**, and **department_name**.
Use an **INNER JOIN** between **employees** and **departments**.
Interview focus: Basic PK–FK relationship and matching rows.

Easy 2 — LEFT JOIN and Missing Relationships

Display every department along with its employees.
Show **department_id, department_name, employee_id, employee_name**.
Departments with no employees must also appear.
Interview focus: Understand why LEFT JOIN is different from INNER JOIN.

Easy 3 — One-to-One Relationship

Display each employee's **employee_name, email, phone, and city** by joining **employees** with **employee_details**.
Return only employees who have a matching details record.
Interview focus: One-to-one relationship using a shared PK.

Medium Level

Medium 4 — Employees and Their Projects

Display **employee_name, project_name, and hours_worked** for every employee-project assignment.
You will need to join **employees** → **employee_projects** → **projects**.
Interview focus: Multi-table joins and many-to-many style bridge tables.

Medium 5 — Employees Without Projects

Find all employees who are not assigned to any project.
Return **employee_id, employee_name, department_id**.
Use a JOIN-based solution rather than a subquery.
Interview focus: LEFT JOIN + NULL logic.

Medium 6 — RIGHT JOIN Practice

Display every department and the employees belonging to it, using a **RIGHT JOIN**.
The result must include departments even when they have no employees.
Return **department_name, employee_id, employee_name**.
Interview focus: Understand how changing the join direction changes which table is preserved.

Hard Level

Hard 7 — Manager Relationship

The **employees** table contains **manager_id**, which references another employee's **employee_id**.
Display **employee_name**, **manager_name**, and **department_name** for employees who have a manager.
Hint: You need to join the employees table to itself and also join departments.
Interview focus: Self-join + foreign key relationship + multi-table join.

Hard 8 — FULL JOIN for Data Reconciliation

Write a query using **FULL JOIN** to compare departments and projects by **department_id**.
Return **department_name, project_id, project_name**.

Your result must show:
• departments with projects
• departments without projects
• projects whose department_id has no matching department
Interview focus: FULL JOIN and unmatched rows on both sides.

Hard 9 — CROSS JOIN Business Logic

The company wants to generate every possible combination of **department** and **project** for a planning exercise.
Use a **CROSS JOIN** to return **department_name** and **project_name** for every possible combination.
Then add a calculated column named **planning_label** that combines them in the format:
Department - Project
Interview focus: Cartesian products and calculated output.

Hard 10 — Final Interview Challenge

Write one query that returns the following for every department:

department_name
employee_count — number of employees in the department
project_count — number of distinct projects associated with the department
employees_without_projects — employees in that department who are not assigned to any project
average_salary — average employee salary rounded to 2 decimal places

Requirements:
1. Departments with zero employees must still appear.
2. Departments with employees but zero projects must still appear.
3. Avoid counting the same employee multiple times because of project joins.
4. Use appropriate JOINs and aggregation.
5. Sort by **employee_count** descending, then **department_name** ascending.

Interview focus: Multi-table joins + LEFT JOIN + COUNT(DISTINCT) + CASE/NULL logic + GROUP BY + ORDER BY.

Interview Practice Rules

• First identify the relationship between the tables before writing the JOIN.

• For every JOIN, ask: Which table's rows must always be preserved?

• Before using COUNT(), check whether a JOIN can duplicate rows.

• For one-to-many relationships, think carefully about whether you need COUNT(*) or COUNT(DISTINCT ...).

• For LEFT/FULL JOIN questions, inspect NULL values to understand unmatched records.

• Try solving each question without looking up the answer. Then explain your query aloud as if you were in an interview.

Recommended challenge: Solve Questions 1–6 without assistance first. For Questions 7–10, focus especially on explaining why you selected each JOIN and how the JOIN affects the number of rows.